

Task :1

Task Name: Create an Abstract Class Shape and Implement area() Method in Circle and Rectangle Classes

Objective:

1. To understand the concept of abstraction in Java.
2. To learn how abstract classes and abstract methods work.
3. To implement abstract methods in subclasses.
4. To calculate the area of different shapes using abstraction.

Problem Analysis:

Abstraction is one of the core principles of Object-Oriented Programming (OOP). It hides implementation details and shows only the essential features of an object. In this task, an abstract class named Shape is created with an abstract method area(). The subclasses Circle and Rectangle inherit the abstract class and provide their own implementations of the area() method. This demonstrates how abstraction allows different classes to perform similar tasks in different ways.

Input:

- Radius of Circle
- Length and Width of Rectangle

Process:

- Create an abstract class named Shape.
- Declare an abstract method area().
- Create a subclass Circle and implement the area() method.
- Create a subclass Rectangle and implement the area() method.
- Create objects of both subclasses.
- Call the area() method to calculate and display areas.

Output:

The program displays the area of a circle and a rectangle.

Algorithm:

1. Start the program.
2. Create an abstract class named Shape.
3. Declare an abstract method area().
4. Create class Circle extending Shape.
5. Implement the area() method in Circle.
6. Create class Rectangle extending Shape.
7. Implement the area() method in Rectangle.
8. Create objects of Circle and Rectangle.
9. Call the area() method for each object.
10. Display the results.
11. End the program.

Source Code:

```
abstract class Shape {  
  
    abstract void area();  
}  
  
class Circle extends Shape {  
  
    double radius = 7;  
  
    void area() {  
        double result = 3.1416 * radius * radius;  
        System.out.println("Area of Circle = " + result);  
    }  
}  
  
class Rectangle extends Shape {  
  
    double length = 10;  
    double width = 5;
```

```

void area() {
    double result = length * width;
    System.out.println("Area of Rectangle = " + result);
}
}

public class TestShape {

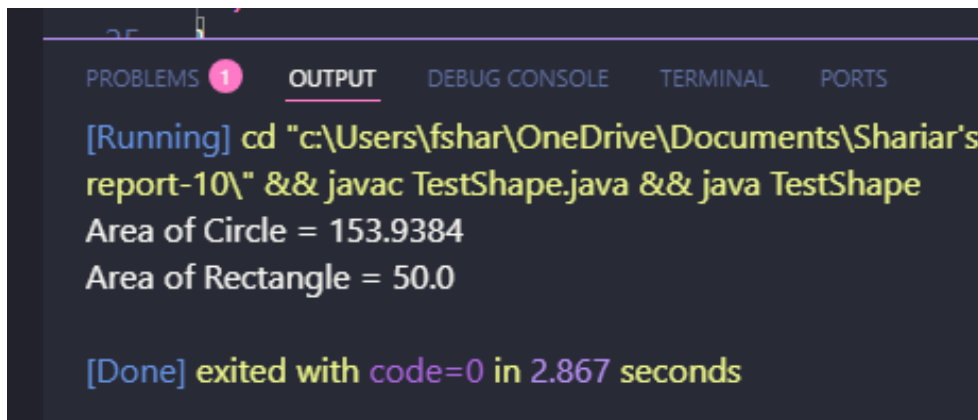
    public static void main(String[] args) {

        Circle c = new Circle();
        Rectangle r = new Rectangle();

        c.area();
        r.area();
    }
}

```

Output:



```

[Running] cd "c:\Users\fshar\OneDrive\Documents\Shariar's report-10\" && javac TestShape.java && java TestShape
Area of Circle = 153.9384
Area of Rectangle = 50.0

[Done] exited with code=0 in 2.867 seconds

```

Discussion:

This program demonstrates abstraction using an abstract class and abstract method. The abstract class Shape provides a common structure, while the subclasses Circle and Rectangle implement the area() method according to their own requirements. Abstraction helps improve code reusability, flexibility, and maintainability.

Task 2

Task Name: Create an Interface ATMSERVICE and Implement It in a Class DBBL

Objective:

1. To understand the concept of interfaces in Java.
2. To learn how to achieve abstraction using interfaces.
3. To implement interface methods in a concrete class.
4. To simulate basic ATM operations using object-oriented programming.

Problem Analysis:

An interface is a blueprint of a class that contains abstract methods. Any class implementing the interface must provide implementations for all its methods. In this task, an interface named ATMSERVICE is created with methods withdraw(), deposit(), and checkBalance(). A class named DBBL implements the interface and provides the functionality for these ATM operations. This demonstrates abstraction and standardization through interfaces.

Input:

- Deposit Amount
- Withdrawal Amount
- Current Balance

Process:

- Create an interface named ATMSERVICE.
- Declare methods withdraw(), deposit(), and checkBalance().
- Create a class DBBL that implements the interface.
- Define all interface methods inside the class.
- Create an object of DBBL.
- Call the methods to perform ATM operations.

Output:

The program displays deposit, withdrawal, and balance information.

Algorithm:

1. Start the program.
2. Create an interface named ATMService.
3. Declare methods withdraw(), deposit(), and checkBalance().
4. Create a class DBBL implementing the interface.
5. Define all interface methods.
6. Initialize account balance.
7. Create an object of the DBBL class.
8. Call deposit().
9. Call withdraw().
10. Call checkBalance().
11. Display results.
12. End the program.

Source Code:

```
interface ATMService {  
  
    void withdraw(double amount);  
  
    void deposit(double amount);  
  
    void checkBalance();  
}  
  
class DBBL implements ATMService {  
  
    double balance = 10000;  
  
    public void withdraw(double amount) {  
  
        if (amount <= balance) {  
            balance = balance - amount;  
            System.out.println("Withdraw Successful: " + amount);  
        } else {
```

```

        System.out.println("Insufficient Balance");
    }
}

public void deposit(double amount) {

    balance = balance + amount;
    System.out.println("Deposit Successful: " + amount);
}

public void checkBalance() {

    System.out.println("Current Balance: " + balance);
}
}

public class TestATM {

    public static void main(String[] args) {

        DBBL account = new DBBL();

        account.deposit(5000);

        account.withdraw(3000);

        account.checkBalance();
    }
}

```

Output:

```

[Running] cd "c:\Users\fsnar\OneDrive\Documents\Shariar's Fold
report-10\" && javac TestATM.java && java TestATM
Deposit Successful: 5000.0
Withdraw Successful: 3000.0
Current Balance: 12000.0

[Done] exited with code=0 in 2.003 seconds

```

Discussion:

This program demonstrates the use of interfaces in Java. The interface `ATMService` defines a standard set of ATM operations, while the class `DBBL` provides their implementations. Interfaces promote abstraction, consistency, flexibility, and code reusability. This approach is widely used in real-world software development to define common behaviors across different systems.